EECS3311 Stage 1 Project Design Report

Oct 8, 2026

Task 1.1: Project Description

The Library Agent is a Java desktop application in which library members find, reserve, borrow, return and rate books by talking to an AI agent in plain language, through a JavaFX GUI or a command-line interface, while librarians manage the catalogue.

Problem: Library systems make members translate what they want into rigid searches and forms. “Something like The Hobbit, not too long and hold it if it's out” becomes several searches, availability checks and a hold request done by hand, and the system never learns what the member enjoyed.

Target users: Library members (standard and student) who search for, reserve, borrow and return books, and librarians who maintain the catalogue and process checkouts, returns and overdue loans.

What the agent can do: It turns plain-language requests into catalogue searches; plans and carries out multi-step tasks such as finding and reserving several books at once and joining hold queues when copies are out; registers new members through conversation; manages reading lists; answers questions about a book using only its catalogue record; records whether members liked what they borrowed and uses that to recommend books and build saved reading lists; handles renewals and returns; suggests genres for new books; and remembers the conversation so follow-ups like “reserve the second one” work.

Why an AI agent is appropriate: Members' requests mix several steps with fuzzy criteria. Fixed forms cannot interpret them, and a plain chatbot cannot act on the library's data. An agent can interpret the request, decide which tools to call and in what order, react to results such as an unavailable copy and explain what it did, while deterministic Java rules still decide what is allowed.

AI/LLM models: Claude Haiku 5.5 through the Anthropic Messages API is the default model, because it is fast, low-cost and designed for chat and short tool tasks. Claude Sonnet 5.5 may be used for the hardest multi-step requests and as the comparison model during testing; early in Stage 2 both run the same requests, and Sonnet becomes the default if Haiku proves unreliable. The model is a configuration setting, and the API key is read from an environment variable.

How the AI interacts with the rest of the system: The LLM never reads or writes the database. `Planner` sends it the conversation and a list of tool descriptions; it replies with either a tool call (a tool name and arguments, using the API's native tool calling) or a final answer. `ResponseParser` turns the reply into a typed decision, `ToolManager` runs the named tool and every tool validates its arguments before calling a deterministic service. Each undoable action (reserve, check out, return, renew or a reading-list change) goes through `CommandHistory`, so it can be undone, and results are fed back to the LLM until it gives a final answer or a step limit is reached.

Overall architecture. A layered Java 21 design. The JavaFX GUI and the CLI are thin views that both call one `AgentController`. Behind it, the agent core (`Planner`, `PromptComposer`, `ResponseParser`, `ToolManager`, `MemoryManager`, `GenreTagger` and the `LLMClient` interface) decides what to do, the domain services (catalogue, circulation, members, reading lists, feedback and notifications) do it and repositories store everything in SQLite through JDBC. In Lecture 10's terms this is an open layered system, since the controller calls both the agent core and the domain and the agent's tools call the domain directly; the bottom layer is a data-oriented repository (a transactional database the services pull from), and each layer is object-oriented inside, making the whole a hierarchical heterogeneous architecture.

Minimum requirements.

Requirement	How the project meets it
GUI	JavaFX application with Chat, Catalog, Account and Librarian tabs covering every feature
CLI	Accepts plain language and short commands (`reserve`, `return`, `renew`, `rate`, `undo`, …) and calls the same `AgentController` as the GUI
At least 10 meaningful features	14 features in Task 1.2; login and logout are not counted
At least 5 design patterns	9: State, Strategy, Composite, Command, Observer, Template Method, Facade, MVC, Adapter
AI model	Claude Haiku 5.5 and Claude Sonnet 5.5
Agent behaviour	Multi-step planning, tool use (12 tools), retrieval of catalogue records, conversation memory and decisions based on availability, holds and ratings

Task 1.2: Feature Specification

The system has 14 features: 7 hybrid, 2 AI-based and 5 deterministic. Login and logout exist but are not counted. Every feature is reachable from both the GUI and the CLI.

F01: Conversational Registration with Reading Preferences

- **Description:** a new user joins by chatting; the agent collects their details and favourite genres and authors, then creates the account and a preference profile that later drives recommendations.
- **User interaction:** Chat tab (“I'd like to sign up”) or the Register form on the start screen; CLI: plain text or `register`.
- **Input:** name, email, member type (standard or student), favourite genres and authors, optional preferred maximum length.
- **Output:** a new `Member` with an ID, stored `ReadingPreferences` and a welcome message.
- **AI involvement:** hybrid. The LLM extracts the fields and asks for missing ones; `MemberService` validates and saves.
- **Expected workflow:** the agent asks follow-up questions until every required field is present, then calls `RegisterMemberTool`, which validates the values and calls `MemberService.register()` and `updatePreferences()`. The agent confirms with the new member ID.
- **Error/alternative cases:** invalid or already-registered email: the tool rejects it and the agent asks again; a genre not in the catalogue: the agent maps it to the nearest existing subgenre or asks; LLM unavailable: the GUI falls back to the plain registration form.

F02: Natural-Language Catalog Search

- **Description:** members describe what they want in plain words; the agent turns the request into a structured `BookQuery` and shows matching books with availability.
- **User interaction:** Chat tab, or the Catalog tab search bar (keyword search works without AI); CLI: plain text or `search <keywords>`.
- **Input:** a free-text request that may mention genre, author, length, mood or “like *X*”.
- **Output:** a list of matching catalogue books with available copies and hold-queue length.
- **AI involvement:** hybrid. The LLM builds the query and summarizes results; `CatalogService` does the matching, including all subgenres through the genre Composite.
- **Expected workflow:** the LLM calls `searchCatalog` with arguments; `SearchCatalogTool` validates them and builds a `BookQuery`; `CatalogService.search()` returns books; the LLM summarizes them; results are stored in `MemoryManager` for follow-up requests.
- **Error/alternative cases:** no matches: the agent says so and suggests loosening one constraint, never inventing titles; vague request (“something good”): the agent asks one clarifying question or uses saved preferences; invalid arguments such as a negative page count: rejected by the tool's validation.

F03: Reading List Management

- **Description:** members create named lists (“Summer reads”) and add, remove or reorder books, by chat or in the GUI.
- **User interaction:** the Reading Lists panel in the Account tab (buttons and drag to reorder); Chat: “add the second one to my summer list”; CLI: `list add <list> <isbn>`.
- **Input:** list name; a book given by title, ISBN or a reference to earlier results.
- **Output:** the updated `ReadingList`, shown in the Account tab.
- **AI involvement:** hybrid. The LLM works out which book and list the user means; `ReadingListService` makes the change. Adding, removing and reordering run as AddToReadingListCommand, RemoveFromReadingListCommand and MoveInReadingListCommand, so each can be undone.
- **Expected workflow:** the LLM resolves the reference using `MemoryManager.lastResults()` and calls `ManageReadingListTool`, which runs an `AddToReadingListCommand` through `CommandHistory`; `ReadingListService.addBook()` saves it and an event refreshes `AccountView`.
- **Error/alternative cases:** list does not exist: the agent offers to create it; book already on the list: no duplicate is added and the user is told; ambiguous reference: the agent asks which book.

F04: Multi-Book Find and Reserve

- **Description:** a single sentence such as “reserve two sci-fi books under 300 pages, whichever are available” makes the agent search, check availability, reserve available copies and offer the hold queue for the rest.
- **User interaction:** Chat tab; the Catalog tab's Reserve button for a single direct reservation; CLI: plain text or `reserve`

`<isbn>`.

- **Input:** a request with criteria and a count; the logged-in member.
- **Output:** one or more `Reservation` objects (ready or queued) and a summary of what was reserved and each queue position.
- **AI involvement:** hybrid. The LLM plans a sequence of tool calls; `CirculationService` enforces availability and limits.
- **Expected workflow:** the agent searches, checks availability for each candidate and reserves the available ones through `CommandHistory`. For unavailable books it offers a place in the queue, then gives a final summary (SD01).
- **Error/alternative cases:** fewer matches than requested: reserves what exists and reports the shortfall; member at the holds limit: the tool returns an error and the agent explains; book already reserved by the same member: rejected; the agent tries to reserve a book not in the catalogue: `ReserveBookTool` rejects the unknown ISBN.

F05: Hold Queue and Availability Notifications

- **Description:** when no copy is free, the member joins a first-come, first-served queue; when a copy is returned, the next reservation becomes Ready and that member is notified.
- **User interaction:** the Holds panel in the Account tab, showing queue position; a notification badge and pop-up in the GUI; a printed notice in the CLI.
- **Input:** a reservation on an unavailable book; a return event.
- **Output:** queue position; a HOLD_READY notification; the returned copy set to Reserved.
- **AI involvement:** deterministic.
- **Expected workflow:** `CirculationService.reserve()` calls `HoldQueue.enqueue()`; on check-in, `nextInLine()` returns the next reservation, which is marked ready, the copy is reserved for it and `NotificationService.publish()` alerts listeners (SD02).
- **Error/alternative cases:** a ready hold not collected within 3 days expires and the next member is promoted; a member cancels: they are removed and positions shift up.

F06: Checkout with Borrowing Limits

- **Description:** an available copy, or one reserved for this member, is lent out, creating a `Loan` with a due date, while enforcing the member's borrowing limit.
- **User interaction:** the Check Out screen in the Librarian tab (member ID and copy ID); CLI: `checkout <memberId> <copyId>`.
- **Input:** member ID and copy ID.
- **Output:** a `Loan` with a due date 21 days out; the copy moves to the On Loan state.
- **AI involvement:** deterministic.
- **Expected workflow:** a `CheckOutCommand` runs `CirculationService.checkOut()`, which checks the member's active loans against `getBorrowLimit()`, calls `BookCopy.checkOut()` (handled by its current state) and creates the loan (SD03).
- **Error/alternative cases:** limit reached (default 5 for standard, 8 for student): refused; copy reserved for another member: refused by `ReservedState`; copy already on loan: refused by `OnLoanState`; unknown IDs: error message.

F07: Book Return (Check-In)

- **Description:** returned copies are checked in, the loan is closed, any late fee is calculated and the next hold is triggered.
- **User interaction:** the Check In screen in the Librarian tab; members at the self-service kiosk can tell the agent “I'm returning these two” in the Chat tab; CLI: `return <copyId>`.
- **Input:** one or more copy IDs, or titles in chat that are matched to the member's open loans.
- **Output:** a closed `Loan`, a late fee if any, a return receipt and possibly a HOLD_READY notification.
- **AI involvement:** deterministic core; hybrid when started through chat, where the LLM works out which loans are meant.
- **Expected workflow:** a `ReturnBookCommand` runs `CirculationService.checkIn()`, which calculates the fee with the member type's policy, checks the copy in and checks the hold queue (SD02).
- **Error/alternative cases:** copy not on loan: rejected; through chat, a book the member never borrowed: `ReturnBookTool` only accepts the member's own open loans and the agent says so.

F08: Due Dates, Renewals, Overdue Detection and Late Fees

- **Description:** tracks due dates, lets members renew, flags overdue loans and calculates fees using the policy for the member's type.
- **User interaction:** Account tab (loans with due dates, Renew button, fees owed); the Overdue list in the Librarian tab; Chat: “can I keep The Hobbit longer?”; CLI: `renew <loanId>`, `overdue`.

- **Input:** loan ID; the current date.
- **Output:** a new due date or a refusal with the reason; the overdue list; fee amounts.
- **AI involvement:** deterministic (a renewal can also be requested through chat via `RenewLoanTool`).
- **Expected workflow:** a `RenewLoanCommand` runs `CirculationService.renew()`, which allows at most 2 renewals and none while someone is waiting, then calls `Loan.renew()`. `findOverdue(today)` checks `isOverdue()` on each open loan, calls `markOverdue()` on the copy and publishes an event. Fees come from `Loan.calculateFee()` with `MemberType.feePolicy()` (SD03, SD07).
- **Error/alternative cases:** renewal refused when a hold is waiting, the maximum is reached or the loan is already overdue; fees are capped at the standard policy's maximum.

F09: Personalized Recommendations and AI-Curated Reading Lists

- **Description:** the agent suggests books, or builds and saves a whole themed reading list, based on the member's preferences, borrowing history and ratings from F14 and explains why each book fits.
- **User interaction:** Chat (“what should I read next?”, “make me a winter-break list, nothing too long”); the “Recommended for you” panel in the Catalog tab; the Generate list button in the Account tab; CLI: `recommend` or plain text.
- **Input:** the member's profile, history and taste profile, plus an optional theme, list size or constraint.
- **Output:** 3 to 5 recommended catalogue books, each with a one-line reason and its availability, or a saved `ReadingList` of catalogue books.
- **AI involvement:** hybrid. `CatalogService.recommendCandidates()` picks real candidates from the taste profile, favouring liked genres and authors and excluding books rated Disliked or already borrowed; the LLM ranks them, writes the reasons and groups them into a theme.
- **Expected workflow:** the LLM calls `RecommendBooksTool`, which reads `FeedbackService.tasteProfile()` and returns candidates; the LLM picks and explains the best ones. For a list, it then calls `CreateCuratedListTool`, which creates the list with `ReadingListService.createList()` and adds each book through `AddToReadingListCommand` (SD08).
- **Error/alternative cases:** new member with no history: uses stated preferences, or asks a question; no candidates: says so; the LLM names a book not in the candidate list: `Planner` only displays books whose ISBNs came from tool results; tracking turned off: only stated preferences are used; a Disliked book proposed for a list: `CreateCuratedListTool` rejects it.

F10: Grounded Book Q&A

- **Description:** members ask about a book (“what is it about?”, “is it suitable for a 13-year-old?”), and the agent answers only from the catalogue record.
- **User interaction:** Chat; the “Ask about this book” button on the Catalog tab's detail pane; CLI: plain text.
- **Input:** a question and a book reference.
- **Output:** an answer based on the catalogue description and metadata, or a clear statement that the record does not say.
- **AI involvement:** AI-based with retrieval through `GetBookDetailsTool`.
- **Expected workflow:** the LLM identifies the book, calls `getBookDetails` and answers from the returned description.
- **Error/alternative cases:** book not in the catalogue: says so; question unrelated to the library: politely declines; description text containing instructions (prompt injection): `PromptComposer` marks tool results as data, not instructions.

F11: Catalog Management with AI Genre Tagging

- **Description:** librarians add and edit books; the agent suggests subgenres from the book's description, and the librarian confirms them.
- **User interaction:** the Add Book form in the Librarian tab with a “Suggest genres” button; CLI: `addbook`.
- **Input:** ISBN, title, author, page count, description, number of copies.
- **Output:** a new `Book` with its `BookCopy` objects, filed under the confirmed subgenres.
- **AI involvement:** hybrid. `GenreTagger` suggests subgenres; the librarian confirms; `CatalogService` saves.
- **Expected workflow:** see SD05.
- **Error/alternative cases:** duplicate ISBN or missing fields: rejected; the LLM suggests a genre not in the hierarchy: filtered out; LLM unavailable: manual selection.

F12: Undo Last Action

- **Description:** reverses the most recent undoable action (reserve, check out, return, renew or adding, removing or reordering a list book), whether it came from the GUI, the CLI or the agent.

- **User interaction:** Undo button in the toolbar; Chat: “undo that”; CLI: `undo`.
- **Input:** none; uses the command history.
- **Output:** the previous state restored and a message naming the undone action.
- **AI involvement:** deterministic.
- **Expected workflow:** `CommandHistory.undoLast()` pops the last command and calls its `undo()` (SD06).
- **Error/alternative cases:** nothing to undo: message; action can no longer be reversed (for example, a reservation already collected): the command reports why.

F13: Conversation Memory and Follow-Up References

- **Description:** the agent remembers the conversation and its last results, so users can say “reserve the second one” or “add those to my list”.
- **User interaction:** Chat tab, where history stays visible; CLI session.
- **Input:** conversation history and the most recent tool results.
- **Output:** later requests resolved against the right books.
- **AI involvement:** AI-based, with deterministic storage in `MemoryManager`.
- **Expected workflow:** each turn, `Planner` loads `MemoryManager.history()` and `lastResults()` into the request built by `PromptComposer`; after the final answer, the new messages are appended.
- **Error/alternative cases:** ambiguous reference (“that one”): the agent asks; very long history: only the recent turns and the last result list are kept; new session: memory starts empty.

F14: Reading Feedback Tracking

- **Description:** the system records what each member borrows and whether they liked it, building a taste profile that F09 uses for recommendations and curated lists. Members can turn tracking off.
- **User interaction:** after a return, a rating prompt (Liked, Neutral, Didn't like, plus an optional comment) in the Account tab's borrowing history; Chat: “I loved The Hobbit” or “I couldn't finish that one”; a tracking switch in Account settings; CLI: `rate <isbn> liked|neutral|disliked`, `tracking off`.
- **Input:** a book the member has borrowed, a rating, an optional comment; the tracking setting.
- **Output:** a stored `BookFeedback` and an updated taste profile (liked and disliked genres and authors).
- **AI involvement:** hybrid. In chat, the LLM recognizes an opinion and turns it into a rating through `RecordFeedbackTool`; `FeedbackService` stores it and calculates the taste profile deterministically.
- **Expected workflow:** from the GUI, `AccountView.rateBook()` calls `AgentController.rateBook()`, which calls `FeedbackService.recordFeedback()`. From chat, the LLM calls `recordFeedback`, and the tool checks the book is in the member's loan history and tracking is on before saving (SD08).
- **Error/alternative cases:** rating a book never borrowed: rejected; tracking off: nothing is stored and the agent explains how to turn it on; vague opinion (“it was interesting”): the agent asks whether they liked it; rating the same book again: the new rating replaces the old one.

Task 3: Feature-to-Design Traceability

Each feature maps to the use case that describes it, the classes and key methods that implement it, the sequence diagram that shows it at runtime and the design patterns it relies on. Use-case and sequence-diagram IDs refer to the UML diagrams submitted for Task 2.

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Conversational registration with reading preferences	Hybrid	UC01 Register Account	ChatView, AgentController, Planner, LLMClient, RegisterMemberTool, MemberService, Repository	chat(), run(), complete(), execute(), register(), updatePreferences()	SD04 Conversational Registration	Facade, Strategy, Template Method
F02	Natural-language catalog search	Hybrid	UC02 Search Catalog	ChatView, CatalogView, AgentController, Planner, PromptComposer, ResponseParser, SearchCatalogTool, CatalogService, GenreComponent	chat(), run(), buildRequest(), parse(), execute(), search(), getBooks()	SD01 Find and Reserve Books	Facade, Strategy, Template Method, Composite
F03	Reading list management	Hybrid	UC03 Manage Reading List	ChatView, AccountView, MemoryManager, ManageReadingListTool, CommandHistory, AddToReadingListCommand, RemoveFromReadingListCommand, MoveInReadingListCommand, ReadingListService	lastResults(), execute(), run(), createList(), addBook(), removeBook(), moveBook()	SD01 Find and Reserve Books (same agent-loop structure)	Command, Template Method, Observer
F04	Multi-book find and reserve	Hybrid	UC04 Find and Reserve Books	ChatView, AgentController, Planner, ToolManager, SearchCatalogTool, CheckAvailabilityTool,	run(), execute(), availableCopies(),	SD01 Find and Reserve Books	Facade, Strategy,

				ReserveBookTool, CommandHistory, ReserveBookCommand, CirculationService, HoldQueue	reserve(), enqueue()		Template Method, Command
F05	Hold queue and availability notifications	Deterministic	UC04 Find and Reserve Books; UC06 Return Book	CirculationService, HoldQueue, Reservation, BookCopy, NotificationService, AccountView, CliNotifier	enqueue(), nextInLine(), markReady(), reserve(), publish(), onEvent()	SD02 Return Book with Hold Notification	Observer, State
F06	Checkout with borrowing limits	Deterministic	UC05 Check Out Book	LibrarianView, AgentController, CommandHistory, CheckOutCommand, CirculationService, Member, BookCopy, CopyState, Loan	checkOutCopy(), checkOut(), getBorrowLimit(), setState()	SD03 Check Out and Renew	State, Command, Facade
F07	Book return (check-in)	Deterministic	UC06 Return Book	LibrarianView, ChatView, AgentController, ReturnBookTool, CommandHistory, ReturnBookCommand, CirculationService, Loan, BookCopy, HoldQueue	checkInCopy(), checkIn(), calculateFee(), nextInLine()	SD02 Return Book with Hold Notification	Command, State, Strategy, Observer
F08	Due dates, renewals, overdue detection and late fees	Deterministic	UC07 Renew Loan and View Account; UC12 Process Overdue Loans	AccountView, LibrarianView, AgentController, RenewLoanTool, RenewLoanCommand, CirculationService, Loan, BookCopy, LateFeePolicy, MemberType	renew(), findOverdue(), isOverdue(), markOverdue(), calculateFee(), feePolicy()	SD03 Check Out and Renew; SD07 Process Overdue Loans	Strategy, State, Command, Observer
F09	Personalized recommendations and AI-curated reading lists	Hybrid	UC08 Get Recommendations and Curated Lists	ChatView, CatalogView, AccountView, Planner, RecommendBooksTool, CreateCuratedListTool, FeedbackService, CatalogService, ReadingListService, CommandHistory, LLMClient	tasteProfile(), recommendCandidates(), createList(), run(), complete()	SD08 Rate a Book and Build a Curated List	Strategy, Template Method, Command
F10	Grounded book Q&A	AI	UC09 Ask About a Book	ChatView, CatalogView, Planner, PromptComposer, GetBookDetailsTool, CatalogService	askAboutSelected(), buildRequest(), execute(), findByIsbn()	SD01 Find and Reserve Books (same agent-loop structure)	Template Method, Facade
F11	Catalog management with AI genre tagging	Hybrid	UC10 Manage Catalog	LibrarianView, AgentController, GenreTagger, PromptComposer, LLMClient, CatalogService, GenreCategory, Repository	suggestGenres(), buildTaggingRequest(), complete(), addBook(), assignGenre()	SD05 Add Book with AI Genre Tagging	Facade, Strategy, Composite
F12	Undo last action	Deterministic	UC11 Undo Last Action	CliApp, CliCommandParser, AgentController, CommandHistory, Command, CirculationService	parse(), undoLast(), undo(), describe(), cancelReservation()	SD06 Undo Last Action from the CLI	Command, MVC
F13	Conversation memory and follow-up references	AI	UC02, UC03, UC04, UC08, UC09 (supports all chat use cases)	Planner, MemoryManager, PromptComposer	history(), append(), lastResults(), buildRequest()	SD01 Find and Reserve Books	Facade
F14	Reading feedback tracking	Hybrid	UC13 Rate a Book	AccountView, ChatView, AgentController, RecordFeedbackTool, FeedbackService, BookFeedback, ReadingPreferences, Repository	rateBook(), recordFeedback(), setTracking(), tasteProfile()	SD08 Rate a Book and Build a Curated List	Template Method, Facade

MVC applies to every feature, since all of them are reached through views that call `AgentController`; it is listed only where it is the main pattern. Likewise, Adapter (ClaudeClient wrapping the Anthropic SDK behind LLMClient) applies to every feature that calls the LLM, alongside Strategy. PromptComposer is a helper that assembles LLM requests, not the Builder pattern.

Task 4: How Each Feature Is Realized

For each feature: the related use case and sequence diagram, the classes involved and their roles, the important methods and how they collaborate at runtime.

F01: Conversational Registration with Reading Preferences

- **Related use case:** UC01 Register Account
- **Related sequence diagram:** SD04 Conversational Registration
- **Classes involved:**
  - `ChatView` collects the user's messages and shows replies.
  - `AgentController` is the single entry point and passes the message to the agent.
  - `Planner` runs the agent loop until registration is complete.
  - `LLMClient` decides which details are missing and when to call the registration tool.
  - `RegisterMemberTool` validates the name, email and member type.
  - `MemberService` creates the member and preference profile.
  - `Repository` stores the member in SQLite.
- **Important methods:** `ChatView.sendMessage()`, `AgentController.chat()`, `Planner.run()`, `LLMClient.complete()`, `RegisterMemberTool.execute()`, `MemberService.register()`, `MemberService.updatePreferences()`
- **Execution:** when the user types “I'd like to join”, `sendMessage()` passes the text to `AgentController.chat()`, which calls

`Planner.run()`. The LLM asks for any missing name, email or member type. Once all are present it returns a `registerMember` tool call; `RegisterMemberTool` validates the values and calls `MemberService.register()` and `updatePreferences()`, which save the member through the `Repository`. The new member ID goes back to the LLM, which writes the welcome message shown in `ChatView`.

F02: Natural-Language Catalog Search

- **Related use case:** UC02 Search Catalog
- **Related sequence diagram:** SD01 Find and Reserve Books
- **Classes involved:**
  - `ChatView` and `CatalogView` accept the request (chat or search bar).
  - `Planner` runs the agent loop.
  - `PromptComposer` describes the available tools to the LLM.
  - `ResponseParser` turns the LLM's reply into a `ToolCall`.
  - `SearchCatalogTool` validates the arguments and builds a `BookQuery`.
  - `CatalogService` finds matching books.
  - `GenreComponent` returns every book under a genre, including subgenres.
- **Important methods:** `AgentController.chat()`, `Planner.run()`, `PromptComposer.buildRequest()`, `ResponseParser.parse()`, `SearchCatalogTool.execute()`, `CatalogService.search()`, `GenreComponent.getBooks()`
- **Execution:** the request reaches `Planner.run()`, which sends it with the tool list built by `PromptComposer`. The LLM replies with a `searchCatalog` call carrying genre, author, page limit and keywords; `ResponseParser` turns it into a `ToolCall`. `SearchCatalogTool` validates the arguments and calls `CatalogService.search()`, which uses `getBooks()` on the genre hierarchy. The matches return to the LLM, which summarizes them, and `MemoryManager` stores them for follow-ups.

F03: Reading List Management

- **Related use case:** UC03 Manage Reading List
- **Related sequence diagram:** SD01 Find and Reserve Books (same agent-loop structure)
- **Classes involved:**
  - `ChatView` and `AccountView` accept the request; `AccountView` shows the updated list.
  - `MemoryManager` supplies earlier results so “the second one” can be resolved.
  - `ManageReadingListTool` validates the list and the book.
  - `AddToReadingListCommand` makes an addition undoable; RemoveFromReadingListCommand and MoveInReadingListCommand do the same for removals and reordering.
  - `CommandHistory` runs and records the command.
  - `ReadingListService` creates lists and adds or removes books.
- **Important methods:** `MemoryManager.lastResults()`, `ManageReadingListTool.execute()`, `CommandHistory.run()`, `ReadingListService.createList()`, `ReadingListService.addBook()`, `ReadingListService.removeBook()`
- **Execution:** the LLM matches the user's reference to an ISBN using `lastResults()` and calls `manageReadingList`. The tool validates the list and book, then runs an `AddToReadingListCommand` through `CommandHistory.run()`, which calls `ReadingListService.addBook()`. A change event refreshes `AccountView`. From the GUI, the Reading Lists panel issues the same command directly.

F04: Multi-Book Find and Reserve

- **Related use case:** UC04 Find and Reserve Books
- **Related sequence diagram:** SD01 Find and Reserve Books
- **Classes involved:**
  - `Planner` repeats the loop until every step of the request is done.
  - `SearchCatalogTool` finds candidate books.
  - `CheckAvailabilityTool` reports free copies and queue length.
  - `ReserveBookTool` requests each reservation.
  - `ReserveBookCommand` and `CommandHistory` record each reservation so it can be undone.
  - `CirculationService` reserves a free copy or adds the member to the hold queue.
  - `HoldQueue` holds waiting reservations in order.
- **Important methods:** `Planner.run()`, `ToolManager.execute()`, `Book.availableCopies()`, `CommandHistory.run()`, `CirculationService.reserve()`, `HoldQueue.enqueue()`

- **Execution:** one request triggers several loop iterations: a search, an availability check per candidate, then a `reserveBook` call for each available book. Each reservation runs as a `ReserveBookCommand`; `CirculationService.reserve()` reserves a free copy, or calls `HoldQueue.enqueue()` when the member accepts a place in the queue. When every step is done, the LLM returns a final answer summarizing what was reserved and each queue position.

F05: Hold Queue and Availability Notifications

- **Related use case:** UC04 Find and Reserve Books; UC06 Return Book
- **Related sequence diagram:** SD02 Return Book with Hold Notification
- **Classes involved:**
  - `HoldQueue` keeps reservations in first-come order.
  - `Reservation` tracks its status (pending, ready, fulfilled, cancelled, expired).
  - `BookCopy` switches to the Reserved state for the next member.
  - `CirculationService` promotes the next reservation after a return.
  - `NotificationService` publishes the HOLD_READY event.
  - `AccountView` and `CliNotifier` receive the event and alert the member.
- **Important methods:** `HoldQueue.enqueue()`, `HoldQueue.nextInLine()`, `Reservation.markReady()`, `BookCopy.reserve()`, `NotificationService.publish()`, `LibraryEventListener.onEvent()`
- **Execution:** when `CirculationService.checkIn()` finishes a return, it calls `HoldQueue.nextInLine()`. If a reservation is waiting, `markReady()` updates it, `BookCopy.reserve()` holds the copy for that member and `NotificationService.publish()` sends a HOLD_READY event to every subscribed listener, so the member sees it in the GUI or CLI.

F06: Checkout with Borrowing Limits

- **Related use case:** UC05 Check Out Book
- **Related sequence diagram:** SD03 Check Out and Renew
- **Classes involved:**
  - `LibrarianView` collects the member ID and copy ID.
  - `AgentController` starts the checkout.
  - `CheckOutCommand` and `CommandHistory` wrap and record the action.
  - `CirculationService` enforces the borrowing limit and creates the loan.
  - `Member` reports its borrowing limit.
  - `BookCopy` and `CopyState` decide whether this copy can be checked out.
  - `Loan` records the checkout and due date.
- **Important methods:** `LibrarianView.checkOutCopy()`, `AgentController.checkOut()`, `CirculationService.checkOut()`, `Member.getBorrowLimit()`, `BookCopy.checkOut()`, `CopyState.checkOut()`
- **Execution:** `checkOutCopy()` calls `AgentController.checkOut()`, which runs a `CheckOutCommand`. `CirculationService.checkOut()` compares the member's active loans with `getBorrowLimit()`, then calls `BookCopy.checkOut()`, which hands the decision to its current state. Only `AvailableState`, or `ReservedState` for this member, switches the copy to `OnLoanState`; a `Loan` due in 21 days is then created and its due date shown.

F07: Book Return (Check-In)

- **Related use case:** UC06 Return Book
- **Related sequence diagram:** SD02 Return Book with Hold Notification
- **Classes involved:**
  - `LibrarianView` (desk) or `ChatView` with `ReturnBookTool` (self-service) start the return.
  - `ReturnBookCommand` and `CommandHistory` wrap and record it.
  - `CirculationService` closes the loan and calculates any fee.
  - `Loan` and `LateFeePolicy` compute the late fee for the member's type.
  - `BookCopy` returns to Available.
  - `HoldQueue` is checked for the next member.
- **Important methods:** `LibrarianView.checkInCopy()`, `AgentController.checkIn()`, `CirculationService.checkIn()`, `Loan.calculateFee()`, `BookCopy.checkIn()`, `HoldQueue.nextInLine()`
- **Execution:** `AgentController.checkIn()` runs a `ReturnBookCommand`. `CirculationService.checkIn()` finds the open loan, calls `calculateFee()` with the member type's policy, calls `BookCopy.checkIn()` and then hands over to the hold-queue logic in F05. Through chat, `ReturnBookTool` accepts only the member's own open loans before running the same command.

F08: Due Dates, Renewals, Overdue Detection and Late Fees

- **Related use case:** UC07 Renew Loan and View Account; UC12 Process Overdue Loans
- **Related sequence diagram:** SD03 Check Out and Renew; SD07 Process Overdue Loans
- **Classes involved:**
  - `AccountView` and `RenewLoanTool` start renewals from the GUI or chat.
  - `LibrarianView` shows the overdue list.
  - `RenewLoanCommand` and `CommandHistory` wrap and record renewals.
  - `CirculationService` applies the renewal rules and finds overdue loans.
  - `Loan` holds the dates and renewal count.
  - `BookCopy` moves to the Overdue state.
  - `LateFeePolicy` (`StandardFeePolicy`, `StudentFeePolicy`) calculates fees.
  - `MemberType` selects the fee policy and borrowing limit.
- **Important methods:** `AccountView.renewSelected()`, `AgentController.renew()`, `CirculationService.renew()`, `Loan.renew()`, `CirculationService.findOverdue()`, `Loan.isOverdue()`, `BookCopy.markOverdue()`, `Loan.calculateFee()`, `MemberType.feePolicy()`
- **Execution:** a renewal runs as a `RenewLoanCommand`; `CirculationService.renew()` refuses if a hold is waiting, two renewals are used or the loan is overdue, and otherwise calls `Loan.renew()`. When the librarian opens the overdue list, `findOverdue(today)` calls `isOverdue()` on every open loan, calls `markOverdue()` on each late copy and publishes an OVERDUE event. Fees always come from `Loan.calculateFee()` using the policy returned by `MemberType.feePolicy()`.

F09: Personalized Recommendations and AI-Curated Reading Lists

- **Related use case:** UC08 Get Recommendations and Curated Lists
- **Related sequence diagram:** SD08 Rate a Book and Build a Curated List
- **Classes involved:**
  - `ChatView`, `CatalogView` and `AccountView` accept requests and show results.
  - `FeedbackService` supplies the member's taste profile.
  - `RecommendBooksTool` asks for real candidate books.
  - `CatalogService` selects candidates from the catalogue.
  - `LLMClient` ranks the candidates, explains them and groups them into a theme.
  - `CreateCuratedListTool` saves the list after checking every book.
  - `ReadingListService` and `CommandHistory` create the list and add each book.
  - `Planner` only displays books that came from tool results.
- **Important methods:** `CatalogView.showRecommendations()`, `FeedbackService.tasteProfile()`, `CatalogService.recommendCandidates()`, `LLMClient.complete()`, `ReadingListService.createList()`, `CommandHistory.run()`
- **Execution:** the LLM calls `recommendBooks`. The tool reads `tasteProfile()` and passes it to `recommendCandidates()`, which favours liked genres and authors and excludes Disliked or already-borrowed books; with tracking off, only stated preferences are used. The LLM picks 3 to 5 books and writes a reason for each. For a list, it then calls `createCuratedList`; the tool checks every ISBN was a candidate, creates the list with `createList()` and adds each book through an `AddToReadingListCommand`.

F10: Grounded Book Q&A

- **Related use case:** UC09 Ask About a Book
- **Related sequence diagram:** SD01 Find and Reserve Books (same agent-loop structure)
- **Classes involved:**
  - `ChatView` and `CatalogView` accept the question (chat or “Ask about this book”).
  - `Planner` runs the agent loop.
  - `PromptComposer` marks tool output as data and instructs the LLM to answer only from it.
  - `GetBookDetailsTool` retrieves the catalogue record.
  - `CatalogService` looks the book up by ISBN.
- **Important methods:** `CatalogView.askAboutSelected()`, `PromptComposer.buildRequest()`, `GetBookDetailsTool.execute()`, `CatalogService.findByIsbn()`
- **Execution:** the LLM identifies the book and calls `getBookDetails`; the tool returns the stored description and metadata from `findByIsbn()`. The LLM answers from that record only. If the record does not cover the question, the agent says so

rather than guessing, and instructions hidden in a description are ignored because `PromptComposer` passes tool output as data.

F11: Catalog Management with AI Genre Tagging

- **Related use case:** UC10 Manage Catalog
- **Related sequence diagram:** SD05 Add Book with AI Genre Tagging
- **Classes involved:**
  - `LibrarianView` collects the book's details and shows suggestions.
  - `AgentController` routes the suggestion and save requests.
  - `GenreTagger` asks the LLM for subgenres and drops any that do not exist.
  - `PromptComposer` builds the tagging request.
  - `LLMClient` proposes subgenres from the description.
  - `CatalogService` validates and saves the book and files it under its genres.
  - `GenreCategory` is the hierarchy the book is filed into.
- **Important methods:** `LibrarianView.addBook()`, `AgentController.suggestGenres()`, `GenreTagger.suggestGenres()`, `PromptComposer.buildTaggingRequest()`, `AgentController.addBook()`, `CatalogService.addBook()`, `CatalogService.assignGenre()`
- **Execution:** `suggestGenres()` builds a tagging request and calls `LLMClient.complete()`; `GenreTagger` keeps only subgenres that exist and returns them to the form. The librarian confirms or edits them, then `AgentController.addBook()` calls `CatalogService.addBook()`, which rejects duplicate ISBNs or missing fields, saves the book and calls `assignGenre()`.

F12: Undo Last Action

- **Related use case:** UC11 Undo Last Action
- **Related sequence diagram:** SD06 Undo Last Action from the CLI
- **Classes involved:**
  - `CliApp` and `CliCommandParser` (or the GUI's Undo button) receive and parse the request.
  - `AgentController` forwards it.
  - `CommandHistory` pops the most recent command.
  - `Command` (for example `ReserveBookCommand`) reverses its own effect.
  - `CirculationService` performs the reversal for circulation actions.
- **Important methods:** `CliCommandParser.parse()`, `AgentController.undoLast()`, `CommandHistory.undoLast()`, `Command.undo()`, `Command.describe()`, `CirculationService.cancelReservation()`
- **Execution:** because every undoable action from the GUI, the CLI and the agent passes through `CommandHistory.run()`, one `undoLast()` call works for all three. It pops the last command and calls its `undo()`; for a reservation, `ReserveBookCommand.undo()` calls `cancelReservation()`. `describe()` supplies the confirmation message, or “Nothing to undo” is returned when the history is empty.

F13: Conversation Memory and Follow-Up References

- **Related use case:** UC02, UC03, UC04, UC08 and UC09 (every chat-based use case)
- **Related sequence diagram:** SD01 Find and Reserve Books
- **Classes involved:**
  - `MemoryManager` stores each session's messages and last results.
  - `Planner` loads memory at the start of every turn and saves it at the end.
  - `PromptComposer` includes the history in the LLM request.
- **Important methods:** `MemoryManager.history()`, `MemoryManager.lastResults()`, `MemoryManager.append()`, `PromptComposer.buildRequest()`
- **Execution:** at the start of `Planner.run()`, `history()` and `lastResults()` are added to the context, so the LLM can resolve references such as “the second one”. After the final answer, `append()` saves the new messages; only recent turns are kept so requests stay within the model's limits.

F14: Reading Feedback Tracking

- **Related use case:** UC13 Rate a Book
- **Related sequence diagram:** SD08 Rate a Book and Build a Curated List
- **Classes involved:**

    - `AccountView` offers the rating prompt and the tracking switch.
    - `ChatView` and `RecordFeedbackTool` turn opinions stated in chat into ratings.
    - `AgentController` handles GUI ratings and the tracking setting.
    - `FeedbackService` stores each `BookFeedback` and builds the taste profile.
    - `ReadingPreferences` records whether tracking is on.
    - `Repository` saves feedback in SQLite.
- **Important methods:** `AccountView.rateBook()`, `AccountView.toggleTracking()`, `AgentController.rateBook()`, `AgentController.setTracking()`, `FeedbackService.recordFeedback()`, `FeedbackService.tasteProfile()`
- **Execution:** from the GUI, `rateBook()` calls `AgentController.rateBook()`, which calls `FeedbackService.recordFeedback()`. From chat, the LLM recognizes an opinion and calls `recordFeedback`; the tool first checks the book is in the member's loan history and that tracking is on. `tasteProfile()` adds up liked and disliked genres and authors from the stored feedback, and F09 reads that profile each time it recommends.

AI Usage and Collaboration

This document was written with the help of generative AI. I planned out the topic and features of the project and presented it to AI, after which the AI gave me the project plan, example, uses and explained the architecture. I used the explanations to draft the document and hence it contains language directly generated by AI but modified by me. In addition, AI was also used to make corrections to the UML diagrams. Model used for this phase of the project was Claude Opus 5.5.